

TP0 · Mapa inicial del backend

1. Del cliente al servidor y de HTTP a REST

1. Explicá las responsabilidades de un cliente y un servidor. Usá una acción cotidiana para describir el recorrido de una solicitud y su respuesta.

El cliente inicia la comunicación. Construye una solicitud, indica que recurso necesita o que acción desea realizar, agrega datos y credenciales si corresponde, envía la solicitud y procesa la respuesta. El servidor recibe la solicitud, valida su sintaxis y permisos, ejecuta la lógica necesaria, consulta o modifica datos y devuelve una respuesta con un código de estado, encabezados y, eventualmente, una representación del resultado.

- Ejemplo cotidiano seria pedir un libro en una biblioteca:

- 1) El cliente solicita un libro.
- 2) El bibliotecario recibe el pedido y verifica si la persona esta habilitada.
- 3) Busca el libro en el catalogo.
- 4) Si lo encuentra, lo entrega.
- 5) Si no esta disponible, informa la situación.

2. Definí API, HTTP y REST y explicá por qué no son sinónimos.

API (Interfaz de programación de aplicaciones o Application programming Interface):

Interfaz mediante la cual un software ofrece operaciones o datos a otro software. Define como utilizar una funcionalidad sin exigir conocer su implementación interna.

HTTP (Protocolo de transferencia de Hipertexto o Hypertext Transfer Protocol):

Protocolo de comunicación de la web. Define métodos, encabezados, código de estado, representación de mensajes y reglas de interacción entre cliente y servidor.

REST(Transferencia de Estado Representacional o Representational State Transfer):

Estilo arquitectónico para sistemas distribuidos. Propone restricciones que organizan la interacción alrededor de recursos, representaciones y una interfaz uniforme.

No son sinónimos:

*Una API puede usar HTTP, pero también gRPC, WebSocket, mensajes o funciones locales.

*HTTP puede utilizarse para una API que no siga REST.

*REST no es un protocolo ni una implementación concreta; es un conjunto de restricciones arquitectónicas.

3. Investiga las restricciones del estilo REST y resumí qué aporta cada una. Incluí un ejemplo de una API HTTP que no pueda considerarse REST.

Las restricciones del estilo arquitectónicos REST, fueron definidas por Roy Fielding en su tesis doctoral. Aportan las siguientes ventajas al desarrollo de software:

- Cliente-Servidor: Separa las responsabilidades de la interfaz de usuario de las de gestión de datos. Aporta independencia de desarrollo, evolución de plataformas por separado y mayor escalabilidad.
- Sin Estado (Stateless): Cada solicitud del cliente al servidor debe contener toda la información necesaria para entenderla y procesarla. El servidor no guarda sesiones del cliente entre peticiones. Aporta escalabilidad, simplifica la visibilidad de las peticiones y facilita el monitoreo.
- Cache (Cacheable): Las respuestas deben etiquetarse implícita o explícitamente como almacenables en cache o no. Aporta eficiencia al reducir la latencia y la carga en el servidor cuando se reutilizan datos previos.
- Interfaz Uniforma (Uniform Interface): Es el pilar central de REST. Simplifica y desacopla la arquitectura mediante cuatro sub-restricciones:
 - 1) Identificación de recursos (URLs).
 - 2) Manipulación de recursos mediante representaciones (JSON, XML).
 - 3) Mensaje auto-descriptivos (headers, metadatos, métodos HTTP adecuados).
 - 4) HATEOS (Hypermedia As The Engine Of Application State): la API guía al cliente mediante enlaces dinámicos.
- Sistema en capas (Layered System): El cliente no puede saber si está conectado directamente al servidor final o a un intermediario (balanceador de carga, proxy, CDN). Aporta seguridad, escalabilidad y abstracción de la infraestructura.
- Código Bajo Demanda (Code on Demand)-Opcional: Permite que el servidor extienda o personalice temporalmente la funcionalidad del cliente enviándole código ejecutable (ej, scripts de JavaScript).

Una API puede utilizar HTTP correctamente y aun así no ser REST. Por ejemplo:

```
POST /api HTTP/1.1
Content-Type: application/json
{ "method": "getUser",
  "params": {
    "id": 42 }
}
```

Este diseño es una API RPC sobre HTTP: concentra operaciones en un endpoint y expresa acciones como getUser, en lugar de identificar recursos mediante una

interfaz uniforme. También podría mantener sesiones implícitas en el servidor y no ofrecer respuestas cacheables ni enlaces navegables. Por eso no debe llamarse REST solamente porque use HTTP y JSON.

2. Principios SOLID en un backend

1. Escribí el nombre completo de cada principio de SOLID, explicalo con palabras propias y acompañalo con un ejemplo breve de backend.

Principio	Nombre completo	Explicacion	Ejemplo de backend
S	Principio de responsabilidad unica	Una clase debería tener un motivo principal para cambiar.	ValidarPedido: Valida datos; ServicioPedido aplica reglas; RepositorioPedido persiste.
O	Principio Abierto/Cerrado	El código debería permitir agregar comportamientos sin modificar continuamente lo ya probado	Agregar 'NotificadorWhatsApp' implementando una interfaz de notificación sin cambiar el servicio de pagos.
L	Principio de sustitución de Liskov	Una implementación derivada debe poder reemplazar a su abstraccion sin romper las expectativas del código cliente	Todo 'Almacenamiento' debe cumplir correctamente guardar y buscar; una implementación de solo lectura no debería presentarse como almacenamiento editable.
I	Principio de segregación de interfaces	Los clientes no deberían depender de métodos que no necesitan	Separar 'LectorUsuario' de 'AdministradorUsuario' en lugar de obligar a todos a implementar ambas operaciones.
D	Principio de inversión de dependencias	La lógica de alto nivel debe depender de abstracciones, no de detalles concretos	'ServicioPago' depende de 'GatewayPago', no directamente de una biblioteca especifica de Stripe o de un proveedor local.

2. Una clase valida solicitudes, aplica reglas de negocio, consulta la base de datos y envía correos. Explica qué principios permitirían mejorarla y por qué aplicar SOLID no significa crear abstracciones para todo.

Principios para mejorar la clase

- SRP (Responsabilidad Unica): La Clase tiene 4 tareas. Hay que dividir las en componentes con un solo rol: un validador, un servicio de negocio, un repositorio (bd) y un notificador (correo).
- DIP (Inversion de dependencias): La lógica de negocio debe conectarse a la BD y al correo mediante interfaces, no instanciando directamente las clases de infraestructura.
- OCP (Abierto/Cerrado): Facilita agregar nuevos cambios (ej. Enviar SMS en lugar de correos) extendiendo el código sin modificar la lógica principal.

¿Por qué SOLID no implica abstraer todo?

Crear interfaces para absolutamente todo genera sobreingeniería. Las abstracciones solo se justifican cuando hay variables reales (múltiples implementaciones) o necesidad de aislamiento para pruebas. Si algo es simple y no va a cambiar, usar abstracciones solo agrega complejidad innecesaria y dificulta la lectura del código.

3. El lenguaje de HTTP

1. Investiga GET, HEAD, POST, PUT, PATCH, DELETE, CONNECT, OPTIONS y TRACE. Resume en una tabla su finalidad, un ejemplo y si son seguros e idempotentes. Aclarará también que existen métodos de extensión.

Metodo	Finalidad	Ejemplo	Seguro	Idempotente
GET	Obtener una representación.	GET /productos/42	Si	Si
HEAD	Obtener los mismos encabezados que GET, sin cuerpo de respuesta	HEAD /manual.pdf	Si	Si
POST	Crear un recurso subordinado o ejecutar una operación no idempotente	POST /pedidos	No	No, normalmente
PUT	Crear o reemplazar completamente el recurso indicado.	PUT /usuario/42	No	Si
PATCH	Aplicar una modificación parcial	PATCH /usuario/42	No	No necesariamente

DELETE	Eliminar el recurso indicado	DELETE /usuarios/42	No	Si
CONNECT	Establecer un tuner hacia el destino, comúnmente para HTTPS mediante un proxy	CONNECT api.ejemplo.com:443	No	No
OPTIONS	Consultar las opciones o capacidades de un recurso	OPTIONS /productos	Si	Si
TRACE	Devolver la solicitud recibida para diagnóstico	TRACE /	Si	Si

Las clasificacion es semántica. Un servidor podría implementar incorrectamente un método, pero eso no cambia el significado definido por HTTP. También existen métodos de extensión, registrados o definidos por aplicaciones y estándares adicionales

2. ¿Se debería usar GET para modificar información en una base de datos? Explicá por qué considerando caché, reintentos, *prefetching* y rastreadores automáticos.

No debería utilizarse GET para modificar una base de datos. GET es seguro y esta destinado a lectura. Si se usa para modificar información, intermediarios y clientes pueden realizar operaciones inesperadas:

- Un navegador o proxy podría almacenar la respuesta en cache.
- Un cliente podría repetir la solicitud automáticamente.
- Un navegador o aplicación podría hacer prefetch de enlaces.
- Rastreadores y robots pueden visitar URLs sin conocer sus efectos.
- Historiales, analizadores, validadores o herramientas automáticas podrían solicitar el recurso.

Por ejemplo, GET /eliminar-cuenta?id=42 podría ejecutarse al ser rastreado. Para una modificación se debe usar POST, PUT, PATCH o DELETE, según la semántica de la operación. Además, una operación sensible debería requerir autenticación, autorización y protección contra solicitudes forjadas cuando corresponda.

3. Explicá las familias 1xx a 5xx y el uso de los códigos más comunes: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500 y 503.

RFC 9110 define cinco familias:

- 1xx --- Informativos: la solicitud fue recibida y el procesamiento continúa.
- 2xx --- Éxito: la solicitud fue recibida, entendida y aceptada.

- 3xx --- Redirección: el cliente debe realizar otra acción para completar la solicitud.
- 4xx --- Error del cliente: la solicitud es inválida, carece de credenciales, no esta permitida o no puede cumplirse.
- 5xx --- Error del servidor: el servidor no puede procesar correctamente una solicitud aparentemente válida.

Códigos frecuentes:

Código	Uso habitual
200 OK	Solicitud exitosa con una representación o resultado.
201 Created	Se creó un recurso; suele incluir 'Location'.
204 No Content	Operación exitosa sin cuerpo de respuesta.
400 Bad Request	Sintaxis, parámetros o formato inválidos.
401 Unauthorized	Falta autenticación válida; pese al nombre, significa "no autenticado".
403 Forbidden	El servidor entendió la solicitud, pero rechaza el acceso.
404 Not Found	El recurso no existe o el servidor decide no revelar su existencia.
422 Unprocessable Content	La sintaxis es válida, pero los datos no cumplen reglas semánticas.
429 Too Many Requests	Se excedió un límite de solicitudes; puede incluir 'Retry-After'.
500 Internal Server Error	Error inesperado del servidor.
503 Service Unavailable	Servicio temporalmente no disponible, por saturación o mantenimiento.

4. Estado, identidad y permisos

1. Comparará aplicaciones stateless y stateful: dónde conservan el estado, ventajas, costos y un ejemplo de cada una. Aclarará si un backend stateless puede consultar una base de datos.

Aspecto	Stateless	Stateful
Ubicación del estado	En el cliente, en cada solicitud o en un almacenamiento externo consultado explícitamente.	En memoria o sesión asociada al servidor.
Ventajas	Escalabilidad horizontal, balanceo sencillo y menor dependencia de una instancia.	Flujo convencional simple y posibilidad de guardar contexto temporal en el servidor.
Costos	Solicitudes potencialmente más grandes y necesidad de resolver revocaciones o consistencia externa.	Afinidad de sesión, replicación, almacenamiento compartido y mayor dificultad al reemplazar instancias.
Ejemplo	API que recibe un token firmado en cada solicitud.	Aplicación que guarda 'session-id' y carrito en una sesión del servidor.

Un backend Stateless SI puede consultar una base de datos. “Sin estado” no significa “sin datos” ni “sin persistencia”; significa que el servidor no depende de recordad el contexto conversacional de una solicitud previa en una instancia concreta. Puede consultar usuarios, permisos, productos o sesiones persistidas.

2. Diferenciá autenticación y autorización mediante un ejemplo con roles y permisos.

La autenticación responde “¿Quién sos?”. La autorización responde “¿Que podés hacer?”.

EJEMPLO:

- Rodrigo inicia sesión y demuestra su identidad con su contraseña.
- El servidor lo autentica como ‘Rodrigo’.
- Su cuenta tiene el rol de ‘Editor’.
- La política permite a ‘Editor’ modificar artículos, pero no administrar usuarios.
- Rodrigo puede ejecutar “PATCH /artículos/10”, pero recibe 403 al llamar “DELETE /usuarios/20”.

3. Explicá las partes de un JWT, qué protege la firma y por qué Base64URL no cifra su contenido. Indicá qué debería validar el servidor y una dificultad de revocar estos tokens.

Un JWT (JSON Web Token) firmado suele tener 3 partes separadas por puntos:

“Base64URL(header).Base64URL(payload).Base64URL(signature)”

- Header: indica el tipo de token y metadatos criptográficos, como ‘alg’ y eventualmente ‘kid’.
- Payload: contiene claims, por ejemplo sub, iss, aud, exp, iat y roles.
- Firma: se calcula sobre el header y el payload codificados; permite detectar modificaciones y verificar que el token fue emitido por quien controla la clave.

RFC 7519 contempla JWT firmados y también JWT cifrados; el JWT firmado habitual no oculta el contenido. Base64URL es una codificación para transportar bytes en URLs y encabezados, no un mecanismo de cifrado. Cualquiera que posea el token puede decodificar el header y payload.

El servidor debería validar, entre otros puntos:

- Firma y clave esperada.
- Algoritmo permitido, evitando aceptar algoritmos no autorizados.
- Emisor (iss).
- Audiencia (aud).

- Expiracion (exp) y, según el caso nbf e iat.
- Identidad (sub).
- Roles, scopes y permisos.
- Revocacion o estado de sesión cuando el sistema lo requiera.

Una dificultad importante es la revocación: un JWT valido normalmente seguirá siendo aceptado hasta su vencimiento, aunque el usuario haya cerrado sesión o perdido permisos. Se puede reducir el problema con tokens de corta duración, refresh tokens rotativos, una lista de revocación por jti o un mecanismo de estado adicional.

5. Control de tráfico y reintentos seguros

1. Explicá qué problema resuelve el rate limiting e investigá al menos dos estrategias para aplicarlo.

El rate limiting limita la cantidad de solicitudes que un cliente puede realizar durante un periodo. Protege disponibilidad, costos y recursos; también ayuda frente a abuso, fuerza bruta y ciertos ataques de denegación de servicio. Puede aplicarse en un proxy, API Gateway o en la propia aplicación.

Estrategias habituales:

- Venta Fija: cuenta solicitudes e intervalos definidos, por ejemplo, de 60 segundos. Es simple, pero permite ráfagas cerca del límite entre dos ventanas.
- Ventana deslizante: considera las solicitudes recientes dentro de los últimos 60 segundos. Es mas precisa, pero requiere mas memoria o estructuras de datos.
- Token bucket: Acumula tokens a una velocidad fija y cada solicitud consume uno. Permite ráfagas controladas.
- Leaky bucket: Procesa solicitudes a una velocidad estable y forma una cola o rechaza excedentes.

2. Implementá, en cualquier lenguaje o pseudocódigo, un límite de 100 solicitudes por minuto usando la IP del usuario. Incluí el contador con vencimiento, la respuesta 429 y explicá sus límites al ejecutar varias instancias o compartir una IP.


```

const LIMITE = 100;
const VENTANA = 60 * 1000;
const accesos = new Map();

function manejarSolicitud(req, res, next) {
  const ip = req.headers['x-forwarded-for']?.split(',')[0].trim() || req.ip ||
  'unknown';
  const ahora = Date.now();

  if(!accesos.has(ip) || ahora > accesos.get(ip).expira) {
    accesos.set(ip, {contador: 1, expira: ahora + VENTANA});

    return next();
  }
  const registro = accesos.get(ip);
  registro.contador++;

  if(registro.contador > LIMITE) {
    const restanteSegundos = Math.ceil((registro.expira - ahora) / 1000);

    return res.status(429)
      .set('Retry-After', String(restanteSegundos))
      .json({
        error: "rate_limit_exceeded",
        message: "Se supero el limite de 100 solicitudes por minuto"
      });
  }
  return next();
}

```

Limitaciones:

- Varias instancias: Al usar memoria local (Map), cada servidor lleva un conteo independiente. Si la app escala a 3 servidores balanceados, un usuario puede realizar hasta 300 peticiones/min antes de ser bloqueado (100 en cada instancia).
 - IP compartida (NAT/Oficinas): Múltiples usuarios en la misma red o red móvil salen a internet con la misma IP pública. Si un usuario agota las 100 peticiones, el error 429 bloquea a todos los demás usuarios de esa misma red.
3. Definí idempotencia, analizá cómo se comportan GET, PUT, DELETE, POST y PATCH, y explicá el uso de Idempotency-Key al reintentar un pago.

La idempotencia es la propiedad de una operación que, al ser aplicada varias veces, produce el mismo resultado que si se hubiera aplicado una sola vez.

Metodo	Comportamiento habitual
GET	Idempotente y seguro.
PUT	Idempotente: establecer el recurso en la misma representación produce el mismo estado final.
DELETE	Idempotente: eliminar un recurso ya eliminado mantiene el estado final, aunque la segunda respuesta puede ser 404.
POST	No idempotente por defecto; varias solicitudes pueden crear varios recursos.
PATCH	No necesariamente idempotente; depende de la operación concreta. Un reemplazo parcial puede serlo, un incremento no.

El encabezado Idempotency-Key es un código único (UUID) que envía el cliente al procesar un pago para evitar cobros duplicados en caso de fallas de red.

- Si la red falla: El cliente no sabe si el pago se realizó y reintenta el cobro enviando la misma clave.
- El servidor detecta la clave: Si reconoce que esa clave ya fue procesada, no vuelve a cobrar; solo devuelve la respuesta original confirmando el pago.

Aporta seguridad financiera y permite reintentos automáticos sin riesgo a duplicar transacciones.

6. Contratos que crecen sin romper clientes

1. Explicá la paginación por página, *offset* y cursor. Sin implementar código, escribí un ejemplo de contrato JSON paginado para GET /api/productos, con sus parámetros y metadatos.
 - Por página: usa page y page_size. Es fácil de entender y permite mostrar “pagina3”, pero las inserciones o eliminaciones pueden desplazar elementos entre paginas.
 - Offset: usa offset y limit. Es flexible para saltar posiciones, pero los offsets grandes pueden ser costosos y también sufrir desplazamientos ante cambios concurrentes.
 - Cursor: usa un cursor opaco basado en una posición estable, como created_at e id. Es adecuado para grandes volúmenes y resultados que cambian con frecuencia, aunque no permite saltar fácilmente a una pagina arbitraria.

```

{
  "data": [
    {
      "id": "prod_101",
      "nombre": "Teclado Mecánico RGB",
      "precio": 85.50,
      "disponible": true
    },
    {
      "id": "prod_102",
      "nombre": "Mouse Inalámbrico",
      "precio": 42.00,
      "disponible": true
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "totalItems": 45,
    "totalPages": 5
  },
  "links": {
    "self": "/api/productos?page=1&limit=10",
    "next": "/api/productos?page=2&limit=10",
    "prev": null,
    "first": "/api/productos?page=1&limit=10",
    "last": "/api/productos?page=5&limit=10"
  }
}

```

2. Investiga las formas habituales de versionar una API y explicá cuándo un cambio obliga a crear una nueva versión.

Formas habituales:

- En la ruta: /api/v1/productos
- En un encabezado: Accept: application/vnd.ejemplo.product-v1+json
- En un parámetro de consulta: /productos?version=1
- Mediante negociación de contenido.
- Versionado por dominio o subdominio, por ejemplo v1.api.ejemplo.com

No todo cambio exige una nueva versión. Generalmente son compatibles:

- Agregar campos opcionales.
- Agregar endpoints.
- Agregar valores que los clientes ya tratan como desconocidos/
- Mejorar documentos sin alterarla semántica.

Suelen ser incompatibles:

- Eliminar o renombrar campos.

- Cambiar tipo o significados.
- Cambiar un campo opcional a obligatorio.
- Modificar códigos de error o reglas de autorización de manera que rompan clientes.
- Cambiar el formato de fechas, paginación o envoltorio de respuesta.
- Alterar el significado de una operación existente

Una estrategia de expansión y contracción permite introducir primero la nueva forma, migrar clientes y retirar la anterior.

3. Definí origen, CORS y *preflight*. Indicá qué permitiría comunicar un frontend en <https://app.ejemplo.com> con una API en <https://api.ejemplo.com> y por qué CORS no reemplaza la autenticación.

Un origen esta formado por esquema, host y puerto. Por lo tanto:

- <https://app.ejemplo.com>
- <https://api.ejemplo.com>

son orígenes distintos porque cambia el host.

Cors es un mecanismo basado en encabezados HTTP mediante el cual el servido indica que orígenes pueden ser leídos por JavaScript ejecutado en un navegador.

Un preflight es una solicitud OPTION que el navegador envia antes de ciertas solicitudes cross-origin, especialmente cuando utilizan métodos o encabezados no considerados simples. Incluye, por ejemplo:

```
OPTIONS /api/productos HTTP/1.1
Origin: https://app.ejemplo.com
Access-Control-Request-Method: PATCH
Access-Control-Request-Headers: Authorization, Content-Type
```

La API podria permitirlo con:

```
Access-Contol-Allow-Origin: https://app.ejemplo.com
Access-Contol-Allow-Methods: GET, POST, PATCH
Access-Contol-Allow-Headers: Authorization, Content-Type
Access-Contol-Allow-Credentials: true
```

No debería usar * junto con credenciales. CORS solamente controla si el navegador permite que el frontend lea la respuesta; no autentica usuarios ni autoriza operaciones. La API debe seguir validando tokens, cookie, permisos y políticas de seguridad.

7. Elegir un modelo de datos

1. Comparará bases SQL y NoSQL según estructura, relaciones, consultas, transacciones, consistencia y escalabilidad.

Criterio	SQL	NoSQL
Estructura	Esquema definido, tablas y tipos.	Estructura flexible; documentos, clave-valor, columnas anchas o grafos.
Relaciones	Claves foráneas y joins expresivos.	Pueden modelarse embebidas, duplicadas o mediante referencias; joins menos uniformes según el motor.
Consultas	Lenguaje declarativo estandarizado, normalmente SQL.	Lenguajes y capacidades específicas de cada motor.
Transacciones	Soporte maduro para transacciones y restricciones de integridad.	Varia según el motor; muchos priorizan operaciones distribuidas o agregados.
Consistencia	Habitualmente fuerte y configurable.	Puede ser fuerte, eventual o configurable según el sistema.
Escalabilidad	Excelente escala vertical y horizontal en muchos motores, aunque puede requerir diseño cuidadoso.	Suele facilitar particiones y escala horizontal para determinar patrones

2. Indicá qué enfoque elegirías para un sistema universitario, un registro masivo de eventos y un catálogo con atributos variables. Justificá y explicá si una aplicación puede combinar ambos enfoques.

- Sistema universitario: SQL. Hay estudiantes, carreras, materias, inscripciones, docentes y calificaciones con relaciones fuertes, restricciones e integridad transaccional.
- Registro masivo de eventos: NoSQL orientado a eventos, columnas anchas o almacenamiento distribuido. El volumen, la escritura continua y las consultas por tiempo o partición suelen ser mas importantes que los joins complejos.
- Catálogo con atributos variables: NoSQL documental puede ser conveniente porque distintos productos pueden tener atributos diferentes. También puede combinarse con SQL para precios, inventario, órdenes y relaciones críticas.

Una aplicación si puede combinar ambos enfoques. Por ejemplo, SQL puede almacenar usuarios, pedidos y pagos, mientras un sistema documental o de eventos conserva descripciones variables, telemetria o un índice de búsqueda. La combinación introduce complejidad de sincronización, observabilidad, copiad de seguridad y consistencia; por eso debe justificarse por necesidades concretas, no por moda.

8. Organizar y encontrar datos

1. Explicá qué es un índice, cómo acelera las consultas y qué costos introduce. Incluí los tipos de índices más habituales y un ejemplo de cuándo conviene usar uno.

Un índice es una estructura auxiliar que mantiene valores de una o mas columnas organizados junto con referencias a las filas. Permite evitar un recorrido completo de la tabla cuando la consulta filtra, ordena o busca por esas columnas. PostgreSQL señala que los índices aceleran la recuperación, pero también introducen costos generales y deben utilizarse con criterio.

Costos principales:

- Espacio adicional.
- Trabajo de mantenimiento en INSERT, UPDATE y DELETE.
- Mayor tiempo de escritura.
- Posible elección de un plan peor si el índice no es selectivo.
- Complejidad operacional y de mantenimiento.

Tipos Habituales:

- B-tree: igualdad, rangos y ordenamiento es el índice general mas común.
- Hash: Búsquedas de igualdad en motores que lo soportan con garantías adecuadas.
- Unico: impide duplicados y puede respaldar restricciones.
- Compuesto: combina varias columnas; el orden de las columnas importa.
- Parcial: indexa solo filas que cumplen una condición.
- Con cobertura: incluye columnas adicionales para responder sin acceder a la tabla, según el motor.
- Texto completo: búsquedas lingüísticas en documentos.
- Espacial: coordenadas y geometrías.
- GIST/GIN u otros especializados: estructuras para rangos, arrays, JSON y búsquedas avanzadas, dependiendo el motor.

Ejemplo: Si la consulta frecuente es:

```
SELECT *  
FROM pedidos  
WHERE cliente_id = 42  
ORDEN BY creado_en DESC  
LIMIT 20;
```

Puede convenir un índice compuesto sobre (cliente_id, creado_en DESC), siempre verificando el plan de ejecución y la distribución de datos.

2. Definí primera, segunda y tercera forma normal y BCNF. Explicá qué problemas evita la normalización y cuándo podría aceptarse una desnormalización controlada.

- Primera forma normal (1FN): cada atributo contiene valores atómicos y no grupos repetidos; no se guardan listas arbitrarias dentro de una celda cuando deberían ser filas relacionadas.
 - Segunda forma normal (2FN): Cumple 1FN y cada atributo no clave depende de toda la clave candidata, no solo de una parte. Es especialmente relevante con clave compuestas.
 - Tercera forma normal (3FN): Cumple 2FN y los atributos no clave no dependen transitivamente de otros atributos no clave.
- BCNF: para toda dependencia funcional no trivial $X \rightarrow Y$, X debe ser una superclave. Es más estricta que 3FN.

La normalización reduce:

- Duplicación innecesaria.
- Inconsistencias entre copias.
- Anomalías de inserción.
- Anomalías de actualización.
- Anomalías de eliminación.
- Ambigüedad sobre cuál es el valor correcto.

La desnormalización controlada puede aceptarse cuando se necesita reducir joins, acelerar lecturas, construir informes o alimentar modelos de búsquedas. Debe acompañarse de una estrategia clara de actualización, restricciones, pruebas y monitoreo. Por ejemplo, guardar el total de una factura puede ser válido si se calcula y actualiza dentro de una transacción, conservando además el detalle que permite auditarlo.

9. Evitar trabajo repetido y acercar contenido

1. Elegí una consulta SQL e implementá, en cualquier lenguaje o pseudocódigo, una caché en memoria con *cache-aside*. Incluí clave, *hit/miss*, TTL e invalidación, y explicá brevemente sus limitaciones.

Consulta elegida:

```
Select * from productos where id = 10;
```

Implementación:

```
const cache = new Map();
const TTL_MS = 60*100;

async function consultarBD(id) {
  console.log(`[BD] Ejecutando: select * from productos where id = ${id}`)
  return {id, nombre: 'Teclado mecanico', precio: 15000};
}
```

```

async function obtenerProducto(id) {
  const clave = `producto:${id}`;
  const ahora = Date.now();
  const registro = cache.get(clave);

  if(registro && ahora < registro.expira) {
    console.log(`[CACHE HIT] Clave: ${clave}`);
    return registro.datos;
  }

  console.log(`[CACHE MISS] Clave:${clave}`);
  const product = await consultarBD(id);

  cache.set(clave, {
    datos: product,
    expira: ahora + TTL_MS
  });

  return product;
}

Funtion invalidadorDeCache(id) {
  const clave = `product:${id}`;
  cache.delete(clave);
  console.log(`[CACHE INVALIDADA] Clave: ${clave}`);
}

```

Limitaciones:

- Memoria local acotada: Al usar un Map interno, los datos viven en la ram del proceso de Node.js. Si la aplicación se reinicia o se queda sin memoria, la cache se borra por completo.
- Inconsistencia entre instancias: En un entorno con múltiples servidores/instancias, la invalidación en una instancia no limpia la cache de las demás, sirviendo datos desactualizados (stale data).
- Falta de desalojo (Eviction): Si la base de datos es muy grande y se consultan miles de registros distintos sin invalidarlos explícitamente, la memoria seguirá creciendo hasta saturar el servidor (a menos que se implemente una política como LRU).

2. Definí CDN e identificá servidor de origen, servidores de borde, contenido habitual y beneficio principal.

Una CDN (Red de Distribución de contenido o Content Delivery Network), es una red distribuida que almacena y entrega contenido desde servidores próximos a los

usuarios. El contenido sigue teniendo un servidor de origen, que es la fuente autorizada cuando el objeto no esta en el cache o necesita validación. Los servidores de borde o Edge reciben solicitudes en distintos puntos geográficos y sirven copias almacenadas.

Contenido habitual:

- Imágenes
- Hojas de estilo.
- JavaScript.
- Fuentes.
- Videos y archivos descargables.
- Respuestas publicas cacheables, cuando la política de seguridad lo permite.

El beneficio principal es reducir la distancia entre el usuario y el contenido, disminuyendo latencia y carga en el origen. Las directivas Cache-Control indican a navegadores, paxies y CDNs como reutilizar o validar respuestas.

Fuentes consultadas.

En el punto 1

- https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- <https://www.rfc-editor.org/rfc/rfc9110.html>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

En el punto 2

- <https://www.freecodecamp.org/espanol/news/los-principios-solid-explicados-en-espanol/>

En el punto 3

- <https://www.rfc-editor.org/rfc/rfc9110.html>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

En el punto 4

- <https://www.rfc-editor.org/rfc/rfc7519.html>

En el punto 5

- [https://cheatsheetseries.owasp.org/cheatsheets/Denial of Service Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Denial_of_Service_Cheat_Sheet.html)

- <https://redis.io/docs/latest/commands/incr/>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Idempotency-Key>

En el punto 6

- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Access-Control-Allow-Methods>
- <https://www.rfc-editor.org/rfc/rfc9110.html>
- <https://martinfowler.com/bliki/ParallelChange.html>

En el punto 7

- <https://www.postgresql.org/>
- <https://www.mongodb.com/es/resources/basics/databases/nosql-explained>

En el punto 8

- <https://www.postgresql.org/docs/current/indexes.html>
- <https://www.postgresql.org/docs/current/ddl-constraints.html>
- https://en.wikipedia.org/wiki/Database_normalization

En el punto 9

- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cache-Control>
- <https://developer.mozilla.org/en-US/docs/Glossary/CDN>